



Sandya Mannarswamy

Welcome to 'CodeSport'. Here are some of the solutions to questions we raised in last month's column.

*M*y last column featured a medley of questions on computer architecture, operating systems and algorithms. Congratulations to four of our readers: K Radha Krishnan, S Priya, P Vasudevan and K Lakshmikanth, for getting most of the answers correct. In this column, we will provide answers to some of the questions, leaving the remaining questions to be addressed next month.

Operating system questions

Q We are aware that the set of memory locations that a process is allowed to access is known as the address space of the process. Consider a program that reads an *array* of 10,00,000 integers. Can this program be executed as a process on a machine whose physical memory is only 4,00,000 bytes?

Yes, virtual memory allows an application to pretend that it has access to a much larger memory than what is physically available.

Q We know that operating systems are typically multi-programmed, which means that multiple processes can be active at the same time. Can there be multiple kernels executing at the same time?

Yes. For example, in a virtual machine, multiple kernels execute at the same time, while there exists a host operating system, which controls access to the physical resources, and allows each operating system to think that it is the only one executing.

Q Is it possible to support multiple processes without support for virtual memory?

Yes. Since nobody has answered this question correctly till now, I will keep the question open for now. However, the clue is to think of segmented address spaces.

Q Consider the following code snippet. What signal gets delivered to the process when this code segment is executed?

```
int main(void)
{
    int* array = (int*) malloc(10);
    array++;
    *array = 120000;
    printf("value is %d\n", *array);
}
```

This was a trick question, in that this is a correct piece of code, and does not cause any SIGBUS signal. We declare an array of integers, and then increment the array pointer by 1. Originally, array points to the element *array[0]*, and after the increment, it points to *array[1]*. The question was intended to confuse you about the increment operation of the pointer.

Q Consider the following code snippet, which shows how a process invokes the *fork* and *exec* system calls. Will it output the line 'execve succeeded'? If not, why?

```
int result = fork();
if (result == 0) /* child code */
{
```